Informe de la Actividad 6: Taller Práctico de Repaso y Consolidación en C++ (Recursividad, Búsqueda Secuencial y Punteros)

Federico Mattia, Máximo Orue, Ulises Ferrari y Lucas del Pino

Grupo 03

E.E.S.T. N.º 1 “Eduardo Ader”, Vicente López

Laboratorio de Programación (LPR), 5.º año 3.º, Grupos A-B

Docente: Mansilla Muñoz York Elías

15 de octubre de 2026

Introducción

El presente informe documenta el trabajo realizado en la Actividad 6 de Laboratorio de Programación, un taller práctico de repaso y consolidación en C++ cuyo propósito es afianzar tres conceptos fundamentales de bajo nivel: la recursividad, la búsqueda secuencial en arreglos y el uso de punteros para intercambiar valores (Mansilla Muñoz, 2026). Los tres conceptos se integraron en un único programa de consola, *main.cpp*, que resuelve un reto por cada pilar y muestra sus resultados en pantalla.

El código se organizó en un repositorio llamado *repasogeneral*, con las carpetas *src/*, *docs/* y *capturas/* exigidas por la consigna, y se publicó en GitHub (Orue et al., 2026a) y en GitLab (Orue et al., 2026b). El código fuente completo, junto con el resto de los entregables, puede consultarse en las direcciones indicadas en la sección de Referencias. Además, el proyecto se configuró con la estrategia *Dual-Remote*, que permite subir los cambios a ambos servidores a la vez mediante un único *git push* para disponer de una copia de respaldo.

En la sección de Desarrollo se responden las cinco preguntas del cuestionario técnico de control, explicando con vocabulario técnico por qué el programa se comporta como lo hace a nivel de memoria. La Conclusión resume los aprendizajes y su posible aplicación en el proyecto integrador anual. Al final se listan las referencias utilizadas. A continuación se detallan la distribución de tareas entre los integrantes (Tabla 1) y el control de revisiones del documento (Tabla 2).

Tabla 1

Integrantes del Grupo 03 y Tareas Realizadas

Integrante	Tarea realizada
Federico Mattia	Escritura y prueba de *main.cpp*; creación del repositorio y configuración del *Dual-Remote*.
Máximo Orue	Análisis del Reto 1 (recursividad) y redacción de las respuestas 1 y 2 del

cuestionario.

Ulises Ferrari Análisis de los Retos 2 y 3 (búsqueda y punteros) y redacción de las respuestas 3 y 4.

Lucas del Pino Redacción de los manuales de usuario y del programador, capturas de ejecución, autoevaluación (respuesta 5) y revisión APA.

Tabla 2

Control de Revisiones del Documento (DocVer)

Versión	Fecha	Estado	Descripción
v1.0.0	15/10/2026	Baseline	Aprobado Versión inicial del informe de la Actividad 6.

Desarrollo

El programa fue compilado con g++ mediante el comando g++ src/main.cpp -o src/repasogeneral.exe. La Figura 1 muestra la salida obtenida al ejecutarlo.

Figura 1

Salida del Programa repasogeneral en la Terminal

```
==========================================================
 TALLER INTEGRADOR REPASO - ESTUDIANTE: Maximo Orue
==========================================================

=== RETO 1: SUMA RECURSIVA ===
Suma acumulada de 1 hasta 5: 15

=== RETO 2: BÚSQUEDA EN MEMORIA CONTIGUA ===
 [SISTEMA] Valor 18 hallado en el indice [6]
 Direccion RAM Hexadecimal: 0x7fff17b7e2c8

=== RETO 3: INTERCAMBIO DE CELDAS ===
Valores previos -> X: 100 | Y: 500
Valores actuales -> X: 500 | Y: 100
==========================================================
```

Nota. La dirección hexadecimal del Reto 2 cambia en cada equipo y en cada ejecución.

Pregunta 1. Caso base

¿Qué error crítico ocurre en la memoria RAM si una función recursiva carece de un caso base de corte?

Cada vez que una función se invoca, el sistema reserva en la pila (*stack*) un marco con sus parámetros, sus variables locales y la dirección de retorno (Stroustrup, 2013). Si la función recursiva no tiene caso base, las llamadas nunca se detienen y los marcos se siguen apilando sin que ninguno se libere. Como la pila es un sector de memoria de tamaño limitado, termina por agotarse y se produce un desbordamiento de pila (*stack overflow*), tras lo cual el sistema operativo finaliza el programa de forma abrupta.

En sumaRecursiva, la línea if (n <= 0) return 0; cumple esa función. Sin ella, el parámetro *n* seguiría disminuyendo (4, 3, 2, 1, 0, -1, -2…) y la función se llamaría indefinidamente. La Tabla 3 muestra cómo se resuelve sumaRecursiva(3): las llamadas se apilan hasta llegar al caso base y luego cada una devuelve su resultado a la anterior.

Tabla 3

Resolución de sumaRecursiva(3) en la Pila de Llamadas

Llamada	**Expresión pendiente**	**Valor devuelto**
sumaRecursiva(3)	3 +	
sumaRecursiva(2)	6	
sumaRecursiva(2)	2 + sumaRecursiva(1)	3
sumaRecursiva(1)	1 + sumaRecursiva(0)	1
sumaRecursiva(0)	Caso base: retorna 0	0

Pregunta 2. Eficiencia

¿Por qué la instrucción *break* optimiza el rendimiento del microprocesador al realizar una búsqueda en arreglos?

La búsqueda secuencial compara el valor buscado con cada celda del arreglo, de a una por vez, por lo que en el peor caso realiza *n* comparaciones, es decir, tiene complejidad O(n).

La sentencia break interrumpe el ciclo for apenas se encuentra la coincidencia. De esa manera el procesador deja de ejecutar comparaciones, incrementos del índice y lecturas de memoria que ya

no aportan información.

En el programa, el valor 18 se encuentra en el índice 6, de modo que el ciclo realiza 7 comparaciones en lugar de las 10 que habría hecho sin break. La mejora no cambia la complejidad en el peor caso, pero sí reduce el trabajo en los demás casos: si el elemento está en la primera celda se necesita una sola comparación. Además, la variable hallado permite informar si el valor no existe cuando el ciclo termina sin interrupciones.

Pregunta 3. Direccionamiento

¿Cómo comprobar mediante el operador & que los elementos de un array ocupan celdas contiguas en la RAM?

El operador & devuelve la dirección de memoria de una variable o de una celda del arreglo. Si se imprimen las direcciones &vectorDatos[0], &vectorDatos[1], &vectorDatos[2], etc., se observa que cada una supera a la anterior en 4 bytes, que es el tamaño de un int en este entorno. Esa diferencia constante y sin saltos demuestra que los elementos están en posiciones consecutivas (Kernighan y Ritchie, 1988). Una forma de verificarlo es agregar al programa:

```
for (int i = 0; i < TAM; i++) {
 cout << "vectorDatos[" << i << "] -> " << &vectorDatos[i] << endl; }
```

La Tabla 4 muestra las direcciones que resultan de esta comprobación, calculadas a partir de la dirección impresa por el programa para el índice 6 en la Figura 1.

Tabla 4

Direcciones de Memoria de los Elementos de vectorDatos

Índice	Valor	Dirección
[0]	12	0x7fff17b7e2b0
[1]	45	0x7fff17b7e2b4
[2]	78	0x7fff17b7e2b8
[3]	90	0x7fff17b7e2bc

[4] 122 0x7fff17b7e2c0

[5] 5 0x7fff17b7e2c4

[6] 18 0x7fff17b7e2c8

[7] 33 0x7fff17b7e2cc

[8] 67 0x7fff17b7e2d0

[9] 99 0x7fff17b7e2d4

Pregunta 4. Variables temporales

¿Qué función cumple la variable temporal en el algoritmo de intercambio de valores con punteros?

La variable temporal conserva una copia del valor de la primera celda antes de que esta sea sobrescrita. Sin ella, la instrucción *ptrA = *ptrB; pisaría el contenido original de *A* y ese dato se perdería, por lo que luego no habría de dónde recuperarlo para asignárselo a *B*. Ambos valores terminarían siendo iguales. La función recibe las direcciones de las variables (intercambiarValores(&x, &y)) y, al desreferenciar con *, modifica las variables originales de main y no copias locales. La Tabla 5 detalla el proceso con los valores del programa.

Tabla 5

Seguimiento del Intercambio de x e y con Punteros

Paso Instrucción x y temporal Inicio — 100 500 — 1 int temporal = *ptrA; 100 500 100 2 *ptrA = *ptrB; 500 500 100 3 *ptrB = temporal; 500 100 100 **Pregunta 5. Autoevaluación**

¿Cuál de los 3 retos te resultó más complejo de comprender a bajo nivel y de qué manera lograste resolverlo?

Para el grupo, el reto más complejo fue el tercero, el intercambio con punteros, porque obliga a distinguir entre la dirección de una variable y el valor que contiene, y a entender que la función modifica las variables originales y no copias. Para resolverlo se hizo un seguimiento paso a paso como el de la Tabla 5, dibujando las celdas de memoria de x, y y temporal después de cada instrucción, y se probaron variantes del programa imprimiendo los valores y las direcciones antes y después de la llamada. El Reto 1 también requirió un esfuerzo particular para visualizar cómo se apilan y se desapilan las llamadas, algo que se aclaró con la traza de la Tabla 3.

Conclusión

La actividad permitió consolidar tres herramientas básicas de la programación en C++. La recursividad requiere un caso base que garantice la finalización de las llamadas y evite el desbordamiento de la pila. La búsqueda secuencial, con complejidad O(n), se optimiza al interrumpir el ciclo con break cuando se halla el elemento. Los punteros permiten trabajar directamente sobre las variables originales mediante el paso de direcciones y la desreferenciación, y el intercambio de valores muestra por qué hace falta una variable temporal.

Estos conceptos son aplicables al proyecto integrador anual. Por ejemplo, recorrer un arreglo contiguo es la base para procesar un *buffer* de lecturas de un sensor, y el paso de datos por puntero evita copiar bloques de información entre funciones. En lo organizativo, el uso del repositorio con *Dual-Remote* y del archivo *docs/CHANGELOG.md* permitió registrar las versiones del trabajo y contar con una copia de respaldo del código.

Referencias

Kernighan, B. W., y Ritchie, D. M. (1988). *El lenguaje de programación C* (2.ª ed.). Prentice Hall.

Mansilla Muñoz, Y. E. (2026). *Actividad 6: Taller práctico de repaso y consolidación en C++*

[Consigna de actividad]. E.E.S.T. N.º 1 “Eduardo Ader”.

Mattia, F., Orue, M., Ferrari, U., y del Pino, L. (2026a). *repasogeneral* [Código fuente]. GitHub. https://github.com/maximoorue30-tech/repasogeneral.git

Mattia, F., Orue, M., Ferrari, U., y del Pino, L. (2026b). *repasogeneral* [Código fuente]. GitLab. https://gitlab.com/tmc232546/repasogeneral.git

Stroustrup, B. (2013). *The C++ programming language* (4.ª ed.). Addison-Wesley.